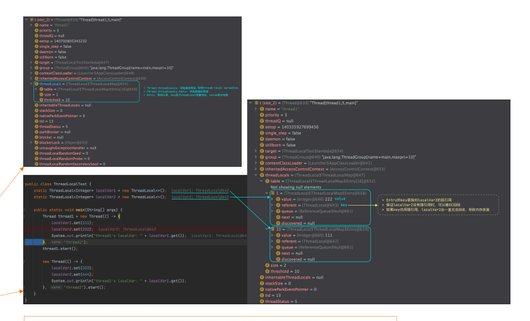

```
t (slot_2) = {Thread@639} "Thread[thread1,5,main]"
  name = "thread1"
  priority = 5
  threadQ = null
  eetop = 140700805343232
  single_step = false
  daemon = false
  stillborn = false
  target = {ThreadLocalTest$lambda@647}
  group = {ThreadGroup@640} "java.lang.ThreadGroup[name=main,maxpri=10]"
  contextClassLoader = {Launcher$AppClassLoader@648}
  inheritedAccessControlContext = {AccessControlContext@649}
  threadLocals = {ThreadLocal$ThreadLocalMap@654}
    table = {ThreadLocal$ThreadLocalMap$Entry[16]@656}
    size = 1
    threshold = 10
  inheritableThreadLocals = null
  stackSize = 0
  nativeParkEventPointer = 0
  tid = 13
  threadStatus = 5
  parkBlocker = null
  blocker = null
  blockerLock = {Object@650}
  uncaughtExceptionHandler = null
  threadLocalRandomSeed = 0
  threadLocalRandomProbe = 0
  threadLocalRandomSecondarySeed = 0
```

> Thread.threadLocals: 定制版哈希表，存储thread-local variables
> Thread.threadLocals.table: 存放数据的数组
> Entry: 数组元素，key是ThreadLocal变量地址，value是本地值

```java
public class ThreadLocalTest {
    static ThreadLocal<Integer> localVar1 = new ThreadLocal<>();   localVar1: ThreadLocal@642
    static ThreadLocal<Integer> localVar2 = new ThreadLocal<>();   localVar2: ThreadLocal@643

    public static void main(String[] args) {
        Thread thread1 = new Thread(() -> {
            localVar1.set(111);
            localVar2.set(222);   localVar2: ThreadLocal@643
            System.out.println("thread1's localVar: " + localVar1.get());   localVar1: ThreadLocal@64
        }, name: "thread1");
        thread1.start();

        new Thread(() -> {
            localVar1.set(333);
            localVar2.set(444);
            System.out.println("thread2's localVar: " + localVar1.get());
        }, name: "thread2").start();
    }
}
```

```
t (slot_2) = {Thread@639} "Thread[thread1,5,main]"
  name = "thread1"
  priority = 5
  threadQ = null
  eetop = 140335927699456
  single_step = false
  daemon = false
  stillborn = false
  target = {ThreadLocalTest$lambda@654}
  group = {ThreadGroup@640} "java.lang.ThreadGroup[name=main,maxpri=10]"
  contextClassLoader = {Launcher$AppClassLoader@641}
  inheritedAccessControlContext = {AccessControlContext@655}
  threadLocals = {ThreadLocal$ThreadLocalMap@647}
    table = {ThreadLocal$ThreadLocalMap$Entry[16]@652}
      Not showing null elements
      1 = {ThreadLocal$ThreadLocalMap$Entry@658}
        value = {Integer@646} 222  value
        referent = {ThreadLocal@643}  key
        queue = {ReferenceQueue$Null@661}
        next = null
        discovered = null
      10 = {ThreadLocal$ThreadLocalMap$Entry@659}
        value = {Integer@660} 111
        referent = {ThreadLocal@642}
        queue = {ReferenceQueue$Null@661}
        next = null
        discovered = null
    size = 2
    threshold = 10
  inheritableThreadLocals = null
  stackSize = 0
  nativeParkEventPointer = 0
  tid = 13
  threadStatus = 5
```

> Entry的key是指向localVar2的弱引用
> 保证localVar2没有强引用时，可以被GC回收
> 如果key也用强引用，localVar2会一直无法回收，导致内存泄漏

> ThreadLocal.nextHashCode，AtomicInteger类型的静态成员变量，用来分配thread-local ID
> 初始值为0，每创建一个ThreadLocal变量，原子性地加一个固定魔数，作为下一个ThreadLocal变量的threadLocalHashCode

> 什么是ThreadLocal?

> **散列方式**：new ThreadLocal()时，自动为其生成threadLocalHashCode
threadLocalHashCode & (len -1) 相当于取后几位作为Entry数组的索引

> **哈希冲突**：线性探测，发生冲突时，依次向右遍历数组，到头时再从0开始遍历，直到找到空位

> **内存泄漏**：ThreadLocal变量回收后，对应的Entry的key为null，如果线程一直不结束，value无法被访问到，也无法被回收，导致内存泄漏
解决方案：每次get/set/remove操作后，扫描并清理key为null的Entry，使用完ThreadLocal后，手动remove

> **变量继承**：创建Thread实例时，如果父线程的inheritableThreadLocals不为空，将其内容copy到自己的inheritableThreadLocals中

```java
public java.lang.Thread() → init() {
  ...
  Thread parent = currentThread();
  if (parent.inheritableThreadLocals ≠ null) {
    新启线程时，如果当前线程的inheritableThreadLocals不为空，把里面的值copy到新线程的inheritableThreadLocals里
    this.inheritableThreadLocals = ThreadLocal.createInheritedMap(parent.inheritableThreadLocals);
  }
  ...
}
```

```java
static ThreadLocalMap createInheritedMap(ThreadLocalMap parentMap)
→ ThreadLocalMap(ThreadLocalMap parentMap) {
  Entry[] parentTable = parentMap.table;
  int len = parentTable.length;
  setThreshold(len);
  table = new Entry[len];
  //把parent.table里的entry一个个复制到新建thread的inheritableThreadLocals.table里
  for (int j = 0; j < len; j++) {
    Entry e = parentTable[j];
    if (e ≠ null) {
      ThreadLocal<Object> key = (ThreadLocal<Object>) e.get();
      if (key ≠ null) {
        Object value = key.childValue(e.value); //e.value
        Entry c = new Entry(key, value); //用parent的key和value创建一个新的entry
        int h = key.threadLocalHashCode & (len - 1);
        while (table[h] ≠ null) h = nextIndex(h, len); //在table里找个合适的位置
        table[h] = c; //放进去
        size++;
      }
    }
  }
}
```

```java
//ThreadLocal使用示例
public class ThreadLocalTest {
  static ThreadLocal<Integer> localVar = new ThreadLocal<>(); //do nothing
  public static void main(String[] args) {
    //启动thread1，设置localVar
    new Thread(() → {
      localVar.set(111); //设置自己的localVar
      System.out.println("thread1's localVar: " + localVar.get());
    }, "thread1").start();

    //启动thread2，设置localVar
    new Thread(() → {
      localVar.set(111);
      System.out.println("thread1's localVar: " + localVar.get());
    }, "thread2").start();
  }
}

//执行之后输出如下，可以看出2个线程使用localVar时，互不影响，各自使用自己的值
thread1's localVar: 111
thread2's localVar: 222
```

```java
public void java.lang.ThreadLocal.set(T value){
  Thread t = Thread.currentThread();
  ThreadLocalMap map = getMap(t); //t.threadLocals
  if (map ≠ null) map.set(this, value); //2.非首次使用，直接存
  else createMap(t, value); //1.首次使用threadLocal，创建map，并保存value
}
```

```java
private void ThreadLocalMap.set(ThreadLocal<?> key, Object value) {
  Entry[] tab = table;
  int len = tab.length;
  int i = key.threadLocalHashCode & (len-1);
  /*线性探测table里所有的entry：
    1).如果找到相同的key，直接更新；
    2).如果找到空key，直接替换；
    3).如果找到空entry，跳出循环 */
  for (Entry e = tab[i]; e ≠ null; e = tab[i = nextIndex(i, len)]) {
    ThreadLocal<?> k = e.get();
    if (k == key) { e.value = value; return;} //1. key已存在，更新value
    if (k == null) { replaceStaleEntry(key, value, i); return; } //2. 有空值，直接替换
  }
  tab[i] = new Entry(key, value); //3. 找到空的位置，新增一个entry
  int sz = ++size;
  if (!cleanSomeSlots(i, sz) && sz ≥ threshold) rehash(); //4. table容量超过75%时，扩容1倍
}
```

```java
void ThreadLocal.createMap(Thread t, T firstValue) {
  t.threadLocals = new ThreadLocalMap(this, firstValue);
}

ThreadLocalMap(ThreadLocal<?> firstKey, Object firstValue) {
    table = new Entry[INITIAL_CAPACITY]; //16
    int i = firstKey.threadLocalHashCode & (INITIAL_CAPACITY - 1);
    table[i] = new Entry(firstKey, firstValue);
    size = 1;
    setThreshold(INITIAL_CAPACITY);
}
```

```java
public T java.lang.ThreadLocal.get(){
  Thread t = Thread.currentThread();
  ThreadLocalMap map = getMap(t); //t.threadLocals
  if (map ≠ null) {
    ThreadLocalMap.Entry e = map.getEntry(this);
    if (e ≠ null) {
      T result = (T)e.value;
      return result;
    }
  }
  return setInitialValue();
}
```

```java
private Entry ThreadLocalMap.getEntry(ThreadLocal<?> key) {
  int i = key.threadLocalHashCode & (table.length - 1); //计算hash地址
  Entry e = table[i];
  if (e ≠ null && e.get() == key) return e; //1. 找到了，直接返回
  else return getEntryAfterMiss(key, i, e); //2. key is not found in its direct hash slot
}

private Entry ThreadLocalMap.getEntryAfterMiss(ThreadLocal<?> key, int i, Entry e) {
  Entry[] tab = table;
  int len = tab.length;
  while (e ≠ null) {
    ThreadLocal<?> k = e.get();
    if (k == key) return e; //2. 找到了，直接返回
    if (k == null) expungeStaleEntry(i); //3. hash地址所指的key为空，删除这个entry（一定程度上防止内存泄漏）
    else i = nextIndex(i, len);
    e = tab[i]; //4. 继续检查下一个entry
  }
  return null; //1. hash地址所指的entry为null，返回空
}
```

```java
private int ThreadLocalMap.expungeStaleEntry(int staleSlot) {
  Entry[] tab = table;
  int len = tab.length;
  //1. 删除这个entry
  tab[staleSlot].value = null; //value引用的对象会在下一次GC时被回收
  tab[staleSlot] = null;
  size—;
  //2. Rehash until we encounter null
  Entry e;
  int I;
  for (i = nextIndex(staleSlot, len); (e = tab[i]) ≠ null; i = nextIndex(i, len)) {
    ThreadLocal<?> k = e.get();
    if (k == null) {
      e.value = null;
      tab[i] = null;
      size--;
    } else {
      int h = k.threadLocalHashCode & (len - 1);
      if (h ≠ i) {
        tab[i] = null;
        while (tab[h] ≠ null) h = nextIndex(h, len);
        tab[h] = e;
      }
    }
  }
  return i;
}
```

```java
private T ThreadLocalMap.setInitialValue() {
  T value = initialValue(); //null
  Thread t = Thread.currentThread();
  ThreadLocalMap map = getMap(t);
  if (map ≠ null) map.set(this, value);
  else createMap(t, value);
  return value;
}
```

```java
public void java.lang.ThreadLocal.remove(){
  ThreadLocalMap m = getMap(Thread.currentThread());
  if (m ≠ null) m.remove(this);
}
```

```java
private void ThreadLocalMap.remove(ThreadLocal<?> key) {
  Entry[] tab = table;
  int len = tab.length;
  int i = key.threadLocalHashCode & (len-1);
  for (Entry e = tab[I]; e ≠ null; e = tab[i = nextIndex(i, len)]) {
    if (e.get() == key) {
      e.clear();
      expungeStaleEntry(i);
      return;
    }
  }
}
```